

The Salesforce logo, consisting of the word "salesforce" in white lowercase letters inside a blue cloud-like shape.

salesforce

Phoenix Secondary Indexes

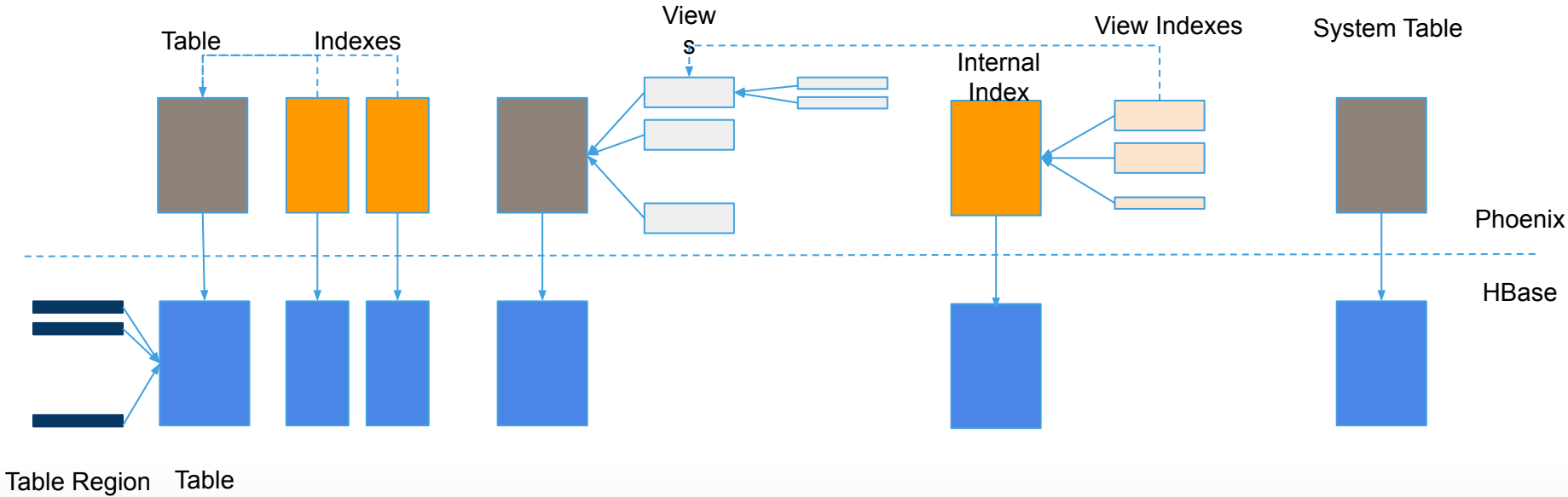
October 2022



Outline

- Background
- CREATE INDEX DDL
- Index State
- Multi-tenant tables and Indexes
- Index Maintainer
- View Indexes
- Query Optimizer
- Strongly Consistent Index Design
 - Challenges
 - Consistency Issues with Concurrent Updates
 - High Level Design
 - Upsert Path for Immutable Indexes
 - Upsert Path for Mutable Indexes
 - Serializing Concurrent Updates
- References

Phoenix/HBase Layering



Phoenix Indexes



```
CREATE TABLE Orders (Order_ID VARCHAR PRIMARY KEY,  
Customer_ID VARCHAR, Order_Date DATE);
```

```
CREATE INDEX Orders_IDX ON Orders  
(Customer_ID) INCLUDE (Order_Date);
```

HBase rowkey

Order_ID	Customer_ID	Order_Date
O_1	C_50	06/01/2018
O_2	C_33	06/02/2018
O_3	C_33	06/03/2018

HBase rowkey - both columns

Customer_ID	Order_ID	Order_Date
C_33	O_2	06/02/2018
C_33	O_3	06/03/2018
C_50	O_1	06/01/2018

```
EXPLAIN SELECT * FROM Orders WHERE Order_ID = 'O_2';
```

```
EXPLAIN SELECT Order_Id FROM Orders WHERE CUSTOMER_ID = 'C_33';
```

CLIENT 1-CHUNK 1 ROWS 176 BYTES PARALLEL 1-WAY
ROUND ROBIN **POINT LOOKUP** ON 1 KEY OVER ORDERS

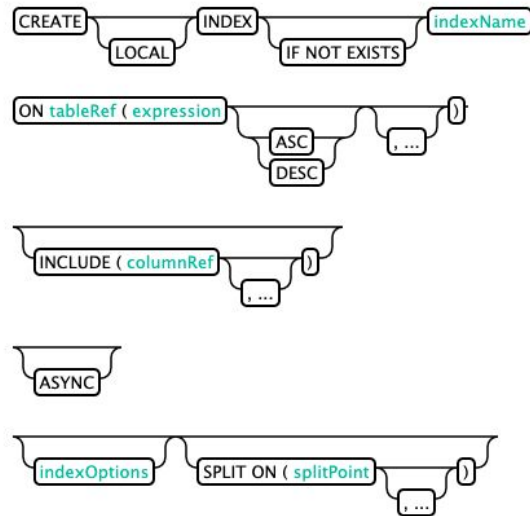
CLIENT 1-CHUNK PARALLEL 1-WAY ROUND ROBIN **RANGE SCAN** OVER
ORDERS_IDX ['C_5'] SERVER FILTER BY FIRST KEY ONLY

Fast!

Fast!

CREATE INDEX DDL

- Local (default Global)
- Specify sort order of secondary index key
- Specify if (any) INCLUDED (covered) columns
- Async (build index in background using MR job)
- Options (TTL, REPLICATION_SCOPE, KEEP_DELETED_CELLS etc...)
 - 4.16 onwards index inherits these options from the parent table



Index States

- PIndexState enum
- Stored as part of index metadata in a syscat column
- BUILDING ('b') (Available for writes, NA for reads)
 - When a new index is created ASYNC, it is in BUILDING state
- ACTIVE ('a') (Available for writes and reads)
- DISABLE('x') (NA for writes and reads)
- There are other states but those are rarely used

Multi-tenant tables and Indexes



```
CREATE TABLE event
(
  org_id   VARCHAR NOT NULL,
  type     CHAR(1)  NOT NULL,
  event_id BIGINT  NOT NULL,
  created_date DATE,
  created_by VARCHAR,
  CONSTRAINT pk PRIMARY KEY (org_id, type, event_id)
)
```

```
CREATE INDEX event_date_idx ON event (created_date)
INCLUDE (created_by);
```

INDEX PK => [created_date, org_id, type, event_id]

```
CREATE TABLE event_mt
(
  org_id   VARCHAR NOT NULL,
  type     CHAR(1)  NOT NULL,
  event_id BIGINT  NOT NULL,
  created_date DATE,
  created_by VARCHAR,
  CONSTRAINT pk PRIMARY KEY (org_id, type, event_id)
) MULTI_TENANT = true;
```

```
CREATE INDEX event_date_idx ON event (created_date)
INCLUDE (created_by);
```

INDEX PK => [org_id, created_date, type, event_id]

Index Maintainer



- Utility class which implements extremely important functionality wrt indexes
- Generate the index row key given a data table row
- Generate the data row key given the index row key
- Generate PUT/DELETE mutations for index updates
- Client serializes this object and transmits to the server
 - **INDEX_PROTO_MD** mutation attribute
 - Separate RPC call using **ServerCacheClient** class
 - Which method is picked depends on the number of mutations in the batch

View Index

- All view indexes share the same physical HBase table (**_IDX_<Base Table>**)
- How do we identify which rows belong to which view index ?
 - **VIEW_INDEX_ID**
 - Generated when the view index is created and stored in the metadata for the index (syscat column)
 - Generated by a sequence one per physical table (sequence name is BaseTable_**ID_**) starting from -32768
 - **VIEW_INDEX_ID** is the leading PK column in the index row key
- On UPSERTS, the **VIEW_INDEX_ID** is available in the **IndexMaintainer** object
- On SELECTS, if the index table is selected, the **VIEW_INDEX_ID** is added to the scan start/stop key.

Query Optimizer



- Determines whether to use index or data table
- If a query projects a column that is not present in the index, the index is not used
- The most row key columns that can be used to form the start/stop scan key
- Use Hints to force usage of table over index or vice-versa
 - NO_INDEX,
 - INDEX (<table_name> <index_name> ...)
 - More in enum Hint in HintNode.java
 - If we force an index to be used and if not all columns are present in the index (uncovered query) we fetch the data row keys from the matched index rows and then join the columns to produce the desired result. This join happens on the server in the latest release
- Run **EXPLAIN <query>** to see the query plan generated to see which table will the query execute against

Strong Consistency Challenge



- Regardless of whether a query is served from a data table or index table, the same result must be returned
- Creating a separate HBase table for each index is not sufficient
- Strong implies updating a data table and its indexes atomically
- Works for both immutable/mutable tables and covered/uncovered indexes

Challenge with Mutable Covered Indexes

- Mutability implies that we need to consider modifying indexed columns too
 - Requires reading data table rows to form index row keys and then deleting existing index rows with invalid row keys

Row Key

ID	Name	City
1234	Alice	Seattle Utah
2345	Bob	San Francisco

Data Table

Row Key

City	ID	Name
San Francisco	2345	Bob
Seattle	1234	Alice
Utah	1234	Alice

Index Table

Consistency Issue with Concurrent Updates

- Concurrent updates on a data row may result in invalid index rows
- Consider row updates {1234, Alice, Utah} and {1234, Alice, Dallas} arrive at the same time

Row Key

ID	Name	City
1234	Alice	Seattle Utah Dallas
2345	Bob	San Francisco

Data Table

Row Key

City	ID	Name
Dallas	1234	Alice
San Francisco	2345	Bob
Seattle	1234	Alice
Utah	1234	Alice

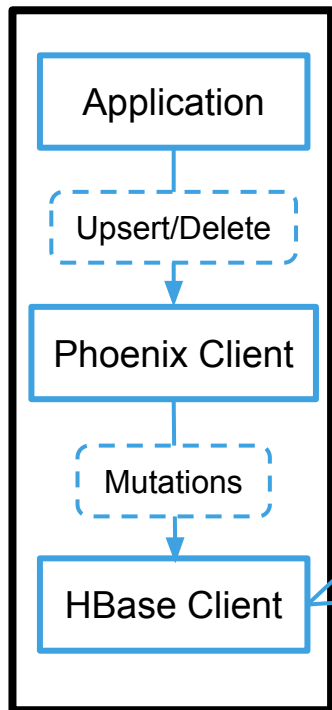
Index Table

High Level Approach

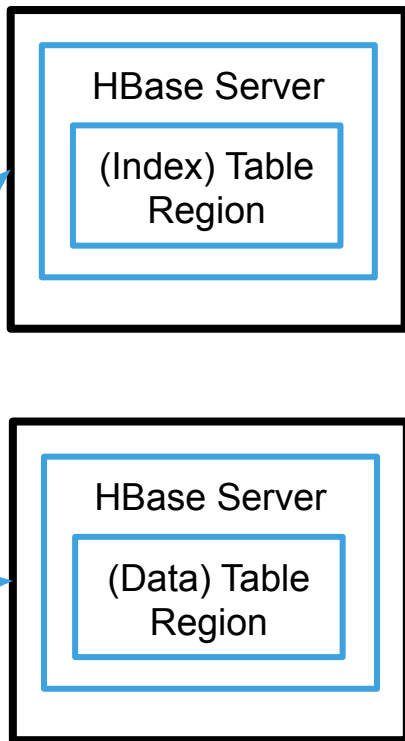
- Upsert : Four-phase approach
 - Read the data table row
 - Insert unverified new versions of index table rows, and update the existing index rows with the unverified status
 - Insert or update the data table row
 - Update the status of the new versions of index table rows to “verified” and delete invalid index rows
- Delete : Four-phase approach
 - Read the data table row and form the index row key(s)
 - Update the status of index table rows to “unverified”
 - Delete the data table row
 - Delete the index table rows
- Read with Index
 - If the index table row is unverified, then repair it using its data table row
 - Repair may mean skip very recently made unverified row, delete the unverified row, or set the status to verified
- Clean up Index
 - Repair unverified index rows
- Consistency
 - Two-phase index commit, read repair, removing invalid index rows are necessary but not sufficient
 - We also need to address the consistency issue due to concurrent updates

Upsert Path for Immutable Indexes

Application Server

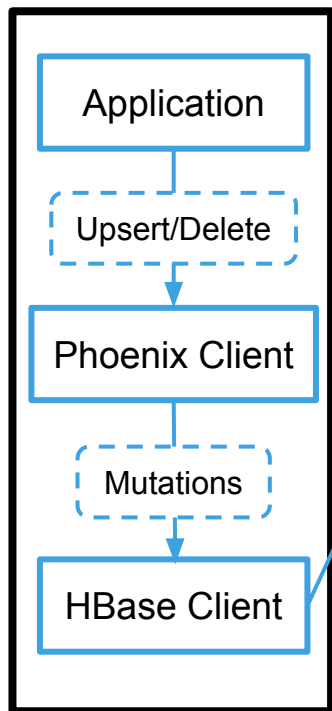


HBase Region Servers

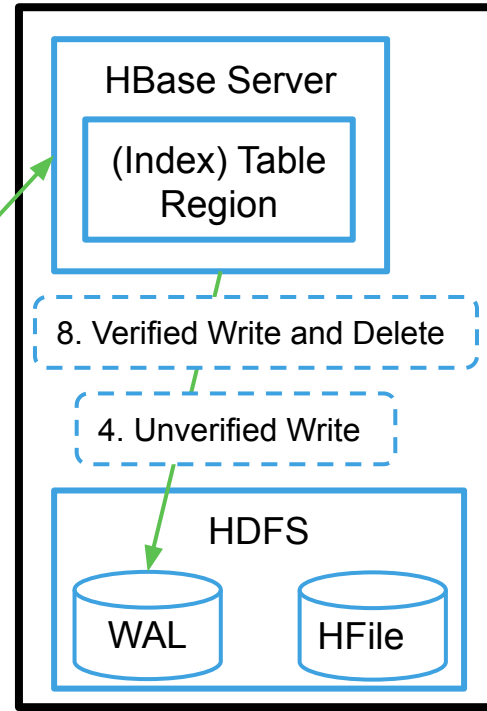
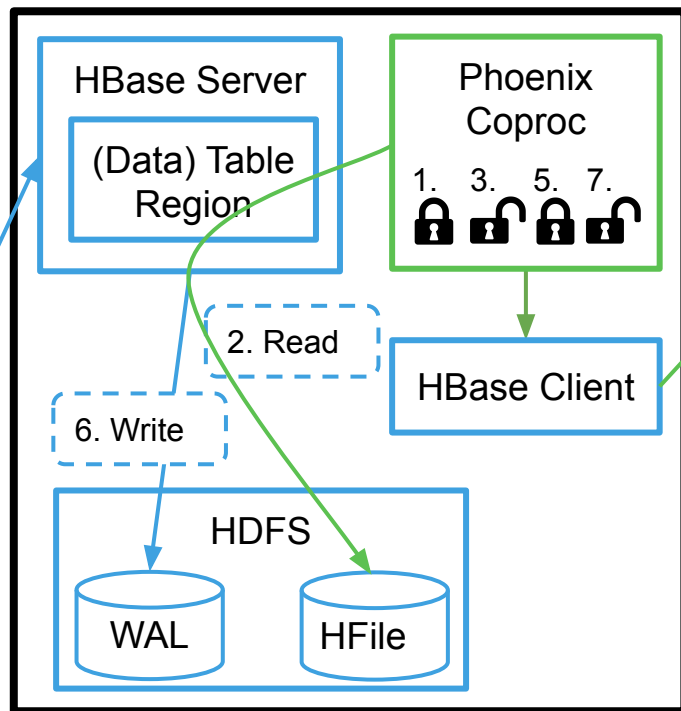


Upsert Path for Mutable Indexes

Application Server



HBase Region Servers



Serializing Concurrent Updates

- Need row locks to serialize updates but cannot do RPCs while holding locks
 - Doing RPCs for index updates while holding locks causes deadlocks
- Instead, use two-phase locking and release the lock before doing RPCs
 - Lock + Read Data Rows + Release + RPCs + Lock + Write Data Rows + Release + RPCs
 - The first phase is to determine the order of concurrent updates
 - The second phase is to ensure the updates to the data table is done orderly
- Maintain after images in memory for in-progress updates
 - For the first update on a row, read the row from disk and create the after image for the update
 - For the next concurrent update, use the row image in memory instead of reading it from disk
- If a data table update fails, then fail the subsequent concurrent updates too
 - Since a data table update failure invalidates the in-memory after image used by the subsequent concurrent updates

Resources

- [Design doc](#) for [PHOENIX-5156](#)
- [Design doc](#) for [PHOENIX-5748](#)
- [Design doc](#) for [PHOENIX-6160](#)
- [Secondary Indexing blog](#) Part 1 and Part 2
- Phoenix OS Secondary Indexing tech talk [Slides](#) and [Recording](#)
- [Phoenix secondary indexing website](#)